

KMI/BEPS - Bezpečnost počítačových systémů

Poznámky z výuky (2025 – 2026)
Verze z 1. září 2000

Vojtěch Netrh
vojtanetrh@gmail.com

Obsah

1	Zkouškové otázky	4
1.1	Hlavní otázky	4
1.2	Vedlejší otázky	4
2	Počítačové sítě	5
2.1	TCP/IP protokol	5
2.2	Autentizace uživatele	5
2.2.1	Autentizace heslem	5
3	Sociální inženýrství	5
4	Kryptografie	6
4.0.1	Praktické aspekty a bezpečnost RSA	6
4.1	Kryptografické hashovací funkce	6
4.2	Digitální podpis	6
4.3	Public key infrastructure	6
5	Certifikáty	6
6	Protokol SSL/TLS	6
6.1	Jak to funguje?	6
6.2	Struktura SSL	6
6.2.1	Record Layer Protocol	6
6.2.2	Change cipher specification protocol	7
6.2.3	Handshake protocol	7
6.2.4	Alert protocol	7
6.3	Zranitelnost SSL	7
6.4	Protokoly využívající SSL/TLS	7
7	Zabezpečení webových aplikací	7
7.1	HTTP basic auth	7
7.2	Nasazení SSL	7
7.3	Bezpečnost při přihlašování/registraci	7
7.3.1	Session hijacking	7
7.3.2	Cross Site Request Forgery	7
7.4	SQL injection	7
8	Škodlivý software (malware)	7
8.1	Kvalita viru	8
8.2	Šíření	8
8.3	Oblíbené cíle virů	8
8.4	Detekce a odstranění malware	8
8.4.1	Taktiky proti detekci	9
8.4.2	Intrusion detection system (IDS)	9
8.4.3	Honeypot	9
8.5	Nedůvěryhodná zařízení	9
9	Chyby a složitost SW i HW	9
9.1	Správa chyb	10
9.2	Typy chyb	10

9.2.1	Buffer overflow	10
9.2.2	Format string attack	10
9.2.3	Integer overflow attacks	10
9.2.4	Command injection attacks	10
9.2.5	Time of check / Time of use attacks	10
9.2.6	XML attacks	11
9.3	Chyby v HW	11
9.3.1	Meltdown	11
9.3.2	Spectre	11
10	Bezpečnost síťových technologií	12
10.1	ARP protokol	12
10.1.1	ARP spoofing	12
10.1.2	MAC flooding	12
10.1.3	Port stealing	13
10.1.4	Další útoky	13
10.2	Protokol PPP a PPTP	13
10.3	Extensible Auth Protocol (EAP)	13
11	Bezpečnost bezdrátových technologií	13

1 Zkouškové otázky

Zkouška proběhne ústní formou. Vylosujete si dvě otázky:

- Hlavní otázka - komplexnější téma, vysvětlíte více do hloubky.
- Vedlejší otázka - menší téma, u kterého vyžadují pouze přehledovou znalost, pochopení principů.

Po vylosování otázky budete mít cca 20 minut na písemnou přípravu, poté budete mít prostor prokázat své znalosti. Otázky označené „Počítačové sítě:“ nebyly probírány v tomto předmětu. Jsou ale natolik důležité pro síťovou bezpečnost, je potřeba je znát a budu je zkoušet.

1.1 Hlavní otázky

1. Počítačové sítě: Architektura TCP/IP (obecně), fyzická a linková vrstva (konkrétně)
2. Počítačové sítě: IP protokol
3. Počítačové sítě: Protokoly TCP a UDP
4. Autentizace uživatele - metody, jednorázová hesla, rekurentní algoritmus
5. Symetrická a asymetrická kryptografie
6. RSA
7. Kryptografické hashovací funkce, digitální podpis
8. Public key infrastructure
9. Email a jeho bezpečnost
10. Protokoly SSL/TLS
11. Zabezpečení webových aplikací
12. Bezpečnost síťových technologií
13. Bezpečnost bezdrátových technologií
14. VPN
15. IPsec
16. Filtrace, proxy, brány
17. RADIUS
18. Škodlivý software a obrana
19. Chyby HW, buffer overflow

1.2 Vedlejší otázky

1. Legislativa - průnik do počítačových systému, osobní údaj
2. Blokové šifry - DES a AES, jejich aplikace
3. Bezpečnost RSA
4. Časová razítka
5. Nevyžádaná pošta, útoky na uživatele, phishing
6. Diffie-Hellmanova výměna klíče
7. SQL injection
8. Prolomení hashovacích funkcí
9. WireGuard
10. Prolomení WEP a WPA
11. Tor
12. OAuth 2.0
13. Eliptické křivky
14. Blockchain

2 Počítačové sítě

2.1 TCP/IP protokol

Běžně se používá dnes v internetu. Architektura má 4 vrstvy (ty využívají služby mezi sebou navzájem). Je nutné definovat **protokoly**, což jsou přesná pravidla podle kterých probíhá komunikace. Původně kladeny nulové nároky na bezpečnost, ty se dodaly až postupně v budoucnu (musela být zajištěna ale i zpětná kompatibilita) např. jako mezivrstvy.

Příklad 1

Rozkouskovaný požadavek počítače, který chce navštívit webovou stránku na nějaké adrese.

2.2 Autentizace uživatele

Proces při kterém se ověřuje uživatel/počítač že je to opravdu on. **Autorizace** je udělení konkrétních práv po tom co proběhne autentizace.

Základní způsoby:

- uživatel zná něco (heslo, hash) unikátního
- uživatel má něco unikátního - soukromý klíč, přístup k jinému zařízení
- biometrika uživatele

V reálném světě se obvykle věří nějaké důvěryhodné autoritě a za padělání hrozí postih (opora v zákoně, ten v technickém světě tak neplatí).

2.2.1 Autentizace heslem

Rozhodující je dnes délka hesla (12-16 znaků je dnes bezpečné). Záleží i na typu útoku (*brute force* vs *rainbow tables*).

Moderní jsou správci hesel (i doporučené).

Dříve se hesla někdy přenášeli v plaintext podobě, dnes už snad vymizelo.

Řešení pro určité případy mohou být i jednorázová hesla. Dříve spíše z pevně daného seznamu (nepraktické – omezená velikost, atd.). Dnes se používá rekurentní algoritmus nebo se předá jiným kanálem (obvykle jako druhý faktor ověření).

Rekurentní algoritmus

Definice 2

Máme **jednosměrnou funkci** f . Obvykle se používají **hashovací funkce**. Zápis $f^n(x)$ značí n -krát použití funkce f s počátečním řetězcem x (řetězení funkce).

1. Inicializace – uživatel zvolí tajný řetězec (*seed*) a tajné číslo n
2. Spočítá $f^n(\text{seed})$ a tuto dvojici odešle
3. [TBA]

Standard **S/KEY** a **OTP**. Tam se to používalo (konkrétně využívalo funkci MD4, která už není bezpečná).

3 Sociální inženýrství

Cílíme na nejvíce zranitelný článek systému, což je člověk. Využívá se psychologie a manipulačních technik. Takže nejde o krádež údajů jako takovou, ale uživatel je sám nedobrovolně

prozradí. Často se na uživatele vyvíjí časový nátlak (neznalý člověk snadněji podlehne). Nejznámější člověk z oboru byl Kevin Mitnick.

4 Kryptografie

[Část která je převzatá od docenta Bartla a je k nalezení v materiálu KMI/KRY.]

Jedná se o tyto tři okruhy:

1. Symetrická a asymetrická kryptografie
2. RSA
3. Kryptografické hashovací funkce, digitální podpis

4.0.1 Praktické aspekty a bezpečnost RSA

Navíc: postranní kanál, poslouchání mikrofonom a měření spotřeby.

4.1 Kryptografické hashovací funkce

4.2 Digitální podpis

4.3 Public key infrastructure

5 Certifikáty

6 Protokol SSL/TLS

Běží na transportní vrstvě (ta je defaultně nezabezpečená). Pro nutnost zabezpečit data na této úrovni vznikly tyto 2 protokoly. Jedná se o dva navzájem nekompatibilní protokoly. Protokol TLS byl historicky první a dnes se již nepoužívá, je nahrazen SSL.

6.1 Jak to funguje?

Jedná se o mezivrstvu mezi aplikační a transportní vrstvou. Architektonicky se jedná o typ *klient-server* (obousměrné). Serverová strana je musí prokázat (autentizovat) certifikátem, pro klienta je to dobrovolné. Šifrování se provádí symetrickou šifrou (kvůli rychlosti). Data z aplikační vrstvy je nutné rozdělit na fragmenty, protože jsou moc velká.

6.2 Struktura SSL

Jeví se jako jediný protokol, ale je složen ze 4 dílčích:

- *Record layer protocol RLP* (manipuluje s daty),
- *Handshake protocol HP* (ustanovuje komunikaci mezi stranami pro RLP),
- *Change cipher specification protocol CCSP* (jediný účel – nastavit budoucí svitu pro HP),
- *Alert protocol AP* (servisní protokol pro předchozí tři)

[Znázornění komunikace na obrázku]

6.2.1 Record Layer Protocol

Rozděluje data na části o max délce 2^{14} . Případně může provádět kompresi. Počítá kontrolní součet a šifruje (na straně příjemce „opačné“ operace). Úvodní data HP jsou nešifrovaná.

V hlavičce RLP se vyskytuje - typ dat (1B), verze (2B), a délka dat (2B).

6.2.2 Change cipher specification protocol

Definuje jedinou zprávu CCS, která dává najevo, že odesílatel přešel na nový způsob šifrování. Může být i uprostřed jediného TCP segmentu.

6.2.3 Handshake protocol

Možnost obnovení relace (rychlejší než navazování nové).

6.2.4 Alert protocol

Slouží ke správě chyb během komunikace. Jsou dva typy – upozornění a fatální chyba (po ní se komunikace končí). Jedná se o 2B zprávy. Domluveno desítky chybových kódů.

6.3 Zranitelnost SSL

6.4 Protokoly využívající SSL/TLS

7 Zabezpečení webových aplikací

Potenciálně výrazně zranitelné, protože jsou dostupné relativně všem a odkudkoliv.

7.1 HTTP basic auth

7.2 Nasazení SSL

7.3 Bezpečnost při přihlašování/registraci

7.3.1 Session hijacking

7.3.2 Cross Site Request Forgery

7.4 SQL injection

8 Škodlivý software (malware)

Ověření fungování SW a že dělá to co chceme. Nikdy to není 100%, ale primitivní metody a pravidla, jejichž dodržování nám dává určitou záruku. Případně ověření podpisu (hashe) aplikace.

Kill switch

Pouze úmyslá „zlá“ funkcionalita. Čili SW dělá něco co uživatel nechce a na uživateli parazituje. Obecně může dělat vše na co má daný software oprávnění (což je často fakt hodně věcí). Např. síťová komunikace, vytěžování zdrojů, zobrazování reklam, odposlech (kamera, mikrofon, klávesy).

Máme několik typů škodlivých softwarů (malware):

1. Vir – program modifikující ostatní programy (většinou šíření sebe sama)
2. Trojský kůň – funkční program, který přidává nechtěnou funkcionalitu navíc
3. Backdoor – skrytá funkcionalita přidána někým od distributora (obvykle obejítí autentizace)
4. Červ – vir šířící své kopie přes síť
5. Spyware – špiónovací software

6. Adware – zobrazování reklam
7. Ransomware – způsobení škody a požadování výkupného (největší otázkou je zda opravdu dodrží slovo)
8. Kontrola počítače – např. botnet
9. Jokeware
10. Kombinace více typů

8.1 Kvalita viru

Několik možných parametrů podle kterých určovat kvalitu.

Špatná detekovatelnost – jak uživatelem, tak i případnými antiviry

Obtížné zneškodnění – změna pozice na disku, tzv. morfovací techniky

Rychlé šíření – jak lokálně, tak po síti

(Ne)náročnost vytvoření a platformní nezávislost – cílení na jedinou platformu vs cílím na vše (to má obvykle vliv i na dobu vývoje, trochu se změnilo s AI)

Užitečnost pro tvůrce – primárně asi peněžní obohacení

8.2 Šíření

Dříve s fyzickými vyměnitelnými médii (už pravěk). Později **.exe** soubory šířené obvykle přes e-mail a skrývání přípon (relativně rychle fixnuto). Dnes se nejběžněji šíří přes chyby v PDF souborech (obvykle to souvisí i s prohlížečem PDF). Také běžně přes sociální inženýrství, že někdo někoho donutí spustit specifický program. Síťové chyby v aplikacích (nejčastěji webové aplikace).

Důležité je, že virus se musí spustit. Když jen sedí na disku je vlastně neškodný. Obvykle viry připojují svůj kód k jinému programu, ideálně ho ještě nějakým způsobem zamaskují. Vhodné reagovat i na skončení dané aplikace, třeba zašifrováním a opětovným uložením na disk.

8.3 Oblíbené cíle virů

Nejllepší možností je *boot sektor*. Když vir zvládne získat kontrolu nad boot sektorem, tak může převzít kompletní kontrolu nad celým počítačem a stává se podstatě nedetekovatelným (znázornění na obrázku jak stojí vrstvy nad sebou). Dalším místem jsou tzv. *resident routines*¹, kde se obvykle jedná o nízké funkcionality jako obsluha kláves nebo ošetření chyb. Další možnosti jsou: napadení knihoven (často používané), samotný OS, makra v Excelu.

8.4 Detekce a odstranění malware

Obecně to nejde, ale snažíme se poučit z minulosti a hledáme **vzorce chování**. Přístup který se vlastně vytváří antiviry. Ustanoví se **vzory (signatury) virů** podle kterých se pracuje. Databáze se musí udržovat co nejvíce aktuální.

Po detekci musíme všechny instance zastavit (obvykle v *nouzovém režimu* OS). Kontrola se ideálně provádí ještě před bootem OS. Po odstranění je dobré najít i cestu, kterou se do systému vir dostal a tu zabezpečit.

¹Funkce OS, které jsou neustále načtené v paměti kvůli rychlosti

Klasifikace virů je složitá a zabývají se tím celé týmy, které je zkoumají a rozdělují do různých kategorií (ty jsou relativně ustálené).

8.4.1 Taktiky proti detekci

V podstatě hra na kočku a myš, obě strany se snaží neustále zlepšovat. Stejně jako všude jinde jsou zlí tvůrci o krok napřed. Viry se snaží mutovat sami sebe, včetně vkládání „zbytečných operací“. Velké zapojení AI u obou stran.

8.4.2 Intrusion detection system (IDS)

Obvykle monitorují síťové chování i konkrétního stroje na základě toho rozhodují. Často se jedná i o celý HW stroj. Kontrolují: konkrétní porty, vytíženost stanic. Používá se strojové učení, které porovnává běžné chování s aktuálním. Mohou provádět i zásahy (vypnutí portu, zabití aplikace).

8.4.3 Honeypot

Falešný uzel, který se snaží nalákat škodlivý software a z jeho působení pak sleduje co dělá a zavádí opatření (i pro celou síť). Řada již hotových řešení, které stačí do sítě implementovat.

8.5 Nedůvěryhodná zařízení

Často se zapomíná na fyzické zabezpečení systému. To nám může zhatit i výborné šifrování a hesla. V dnešní době se dá zaheslovat BIOS či šifrovat celý disk. Dnes i malá zařízení (velikost flash disku) jsou schopná mít obrovskou funkcionalitu, které umožní napadnout a ovládnout systém, v horším případě i celou síť. Příkladem pro tyto zařízení může být *USB killer* či *Rubber ducky*, ale i řada dalších².

9 Chyby a složitost SW i HW

Obecně v dnešní době je SW extrémně komplexní, čímž roste velikost kódu, počet používaných knihoven a kontrola nad takovým množstvím věcí je náročnější. Při používání knihoven a protokolů bychom neměli konat bezmyšlenkovitě a měli bychom alespoň zhruba vědět co to má dělat. Naštěstí i podpůrné nástroje pro programátory jsou lepší a samy pomáhají odhalit řadu chyb.

Bezpečnostní chyba

Definice 3

Chyba v SW, které umožní uživateli dělat to, na co nemá oprávnění.

Máme několik různých variant:

1. Arbitrary code execution (ACE) – spuštění libovolného kódu na počítači
2. Remote code execution – vzdálené spuštění libovolného kódu
3. Eskalace práv – navýšení práv uživatele na **root**
4. Neoprávněný přístup do paměti – aplikace může zapisovat/číst na místě paměti jiných procesů³
5. Útěk z kontejneru – aplikace opustí kontejner (bezpečné prostředí) určený pro běh

²Je možnost mít i hardwarový packet sniffer na ethernetový kabel.

³Mohou se tam nacházet privátní klíče, hesla atd.

Kód využívající chyby v aplikaci pro svůj prospěch.

9.1 Správa chyb

Chyby by se neměly tajit, ale zveřejnit a donutit správce k updatu či tvůrce SW k opravě.

Vznikla databáze a systém označování *Common Vulnerabilities and Exposures (CVE)*, který každé chybě přidělí číslo (např. CVE-2014-0160) a zveřejní bezpečné detaily.

Proces obvykle začíná pouze zveřejněním, že je nějaká chyba, aby se správci mohli připravit na update. Bližší informace (exploity, testy, detaily) se zveřejní až později, aby nemohly být ihned zneužity.

Velké firmy mají tzv. *BugBounty* programy, kde nabízí finanční odměny (poměrně štedré) pokud někdo z veřejnosti chybu objeví a společnosti nahlásí.

9.2 Typy chyb

V této kapitole jsou popsány základní typy chyb společně s problémem který způsobují a případnou obranou proti nim.

9.2.1 Buffer overflow

Pokud v nízkoúrovňovém jazyce nekontrolujeme velikost bufferu, do kterého se něco zapisuje, tak může dojít přepsání paměti jiné aplikace. V lepším případě aplikace jen spadne. V horším je možné spustit libovolný kód nebo skočit jinam do kódu.

Obranou jsou *stack canaries* – přidání náhodného čísla před návratovou hodnotu pro kontrolu nebo použití *NX bitu*, který rozdělí paměť na data a kód.

9.2.2 Format string attack

Formátování řetězce a nekontrolování počtu argumentů způsobí, že číst ze zásobníku nebo na něj dokonce zapisovat.

Obranou je kontrola počtu argumentů.

9.2.3 Integer overflow attacks

V celočíselné aritmetice mají čísla omezenou délku na počet bitů. Při sčítání velkých čísel může hodnota přetéct maximální možnou. To umožní psát do místa na paměti, kam by uživatel neměl.

9.2.4 Command injection attacks

Pokud se v programu přímo volá `shell` může dojít k vložení škodlivého příkazu, který `shell` vykoná. Podobný přístup jako SQL injection attack.

9.2.5 Time of check / Time of use attacks

Případ kdy se časově oddělí kontrola oprávnění a provádění operace. Například při zápisu do specifického souboru.

Obranou je otevření souboru a poté ihned zkontrolovat oprávnění (funkce `fstat()`).

9.2.6 XML attacks

Zpracování XML může být složitější neboť se nejedná o triviální záležitost a v parserech mohou být chyby. Nejtypičtější se využívají **XML entity**, které načtou něco co nikdy nekončí nebo soubory, které by neměly být viděny.

9.3 Chyby v HW

Stejně jako u SW, i zde roste komplexnost a s tím i možnost chyby. Instrukční sady jsou různě emulované a je dokonce možné mít OS přímo na procesoru (viz Intel a Minix).

Problém může nastat pokud jsou chyby pevně svázány s designem HW a nedají se opravit aktualizací mikrokódu.

Poznámka 5

Bezpečnostní chyba v CPU se vůbec tou nejzávažnější chybou.

9.3.1 Meltdown

Pro zvýšení rychlosti se instrukce provádí paralelně (tzv. *out of order*). CPU se pak stará o synchronizaci a předpokládá, že instrukce skončí dobře. Po dokončení obsluhy výjimky můžeme postranním kanálem (konkrétně díky času) zjistit kam do paměti se přistupovalo a data si následně přečíst.

```
1  unsigned char *ptr = /* 0x12..78 */; // adresa hledanych dat
2  unsigned char value = 0; // prectena hodnota
3  unsigned char probe_array[256 * 4096]; // pomocne (velke) pole
4
5  int main() {
6      signal(SIGSEGV, handle_sigsegv); // inicializace
7      fflush();
8      // precte byte z pameti a muze skoncit vyvolanim vyjimky
9      unsigned char t = *ptr;
10     // pristup do probe_array, bude proveden mimo poradi
11     unsigned char x = probe_array[t * 4096];
12     value = t; // pokud nedoslo k vyjimce
13     :read_complete
14     // na toto misto se vrati kod z obsluhy vyjimky
15     printf("Hodnota: %i\n", value);
16     return 0;
17 }
```

9.3.2 Spectre

Velmi podobné jako Meltdown, ale hůř se využívá. Spoléhá se na spekulativní vykonání **if** větví. Pokud se mu spekulace povede, tak se přístup zrychlí, pokud ne tak výsledek zahodí. Jenže i zahozený výsledek je možné v L1 či L2 cache procesoru najít.

Pro provedení musí nejdřív útočník donutit procesor si myslet, že danou instrukci je dobré spekulativně provádět, protože podmínka platí. Poté podstrčí čtení z paměti, kde mohou být uloženy útočníkem chtěná data a procesor spekulativně proveden operaci. Jenže realita je jiná

a podmínka neplatila a data je nutné zahodit. Díky postrannímu kanálu je však možné (podle rychlosti/času) zjistit kam se přistupovalo a data nějakým způsobem z daného místa získat.

Má 2 verze, druhá se zaměřuje na *branch prediction* a *branch target buffer*, kde jsou adresy pro nepodmíněné skoky.

10 Bezpečnost síťových technologií

Ústředním tématem je protokol *ethernet*, který slouží ke komunikaci mezi sousedními uzly a funguje na linkové vrstvě ISO/OSI modelu. Využívá MAC adresy. Rámce se přenáší všesměrově a na síťových kartách je možné využít **promiskuitní režim**, který umožní zachytit a zobrazit rámce, které nejsou mě určené. Rámce mají sice kontrolní součet, ale slouží pouze ke kontrole technických chyb. Útočník může data pozměnit, kontrolní součet přepočítat a nic se nepozná.

Existují 2 varianty: nepřepínaný a přepínaný ethernet. U nepřepínaného jsou uzly spojeny hubem a tím pádem vidí všechno (dnes se již nepoužívá), u přepínaného jsou uzly spojené switchem, který řídí komunikaci mezi nimi a data přeposílá jen tam, kam patří.

10.1 ARP protokol

Pro IP komunikaci musíme logicky znát IP adresu dané síťové karty. Je nutné přiřazení IP a MAC adres, k čemuž slouží **protokol ARP**. Ten používá všesměrový rámce, kde se ptá kdo má danou MAC adresu. Stroj poté odpoví správnou IP adresou.

Poznámka 6

Uzly si udržují vazby v ARP cache. Příkaz `arp`.

Poznámka 7

Odhalení uzlů v síti se dá docílit nástrojem `nmap`.

10.1.1 ARP spoofing

Jde o podvržení MAC adresy routeru (brány) v ARP cache oběti a zároveň podvržení MAC adresy oběti v ARP cache routeru. Tím docílíme MITM útoku, jelikož daná komunikace uzlu s routerem poteče přes nás. Dokonce existuje nástroj **ettercap**, která útok umožní realizovat i s GUI.

Obranou je používat statickou ARP cache či filtrovat odpovědi bez dotazu na speciálních typech switchů. Případně následná kontrola MAC a IP adres u všech paketů – tzv. *dynamic IP lockdown*.

10.1.2 MAC flooding

Switche a routery si udržují CAM (content addressable memory) tabulku, kde jsou vazby MAC adresa a k ní náležící port. Útočník generuje pakety s náhodnými MAC adresami a tím tuto tabulku přeplní a dojde k jejímu resetování. Každý router však může mít reakci mírně odlišnou.

Obranou je detekce IP provozu nebo statické nastavení managovatelného switchu.

10.1.3 Port stealing

Jedná se o MITM útok. Útočník bude simulovat přepojení oběti do jiného fyzického portu (musí dojít k aktualizaci CAM tabulky). Nevýhodou pro útočníka je, že každá komunikace od oběti útok naruší a útočník musí neustále reagovat.

Obranou je možnost statického nastavení CAM nebo aktivní obrana oběti. Otázkou je jak správně odlišit oběť a útočníka.

10.1.4 Další útoky

- ICMP redirect – generování falešných ICMP⁴ zpráv
- DHCP spoofing – vytvoření falešného DHCP serveru v síti
- DNS spoofing – podvržení odpovědi na DNS dotazu

10.2 Protokol PPP a PPTP

Celým názvem *Point to Point Protocol* a *Point to Point Tunneling Protocol*. Tyto protokoly se používají k pro připojení k internetu přes telefonní linku (i dnes – xDSL nebo optika).

- PPP funguje na L2 vrstvě a používá se ke spojení mezi 2 uzly. Umožňuje autentizaci (PAP) a zapouzdření.
- PPTP slouží k vytváření VPN tunelů. Autorem je Microsoft. Používá šifrování MPPE a protokol GRE k přenosu paketů.

Aktuálně je jejich funkcionalita nahrazena jinými a lepšími. Tyto obsahují známe zranitelnosti.

10.3 Extensible Auth Protocol (EAP)

Zajišťuje že zařízení se podle něj budou autentizovat v budoucnu. Nejedná se o jediný autentizační protokol, ale rámec, kde se konkrétní metoda dohodne později.

- EAP-MD5 – obdoba CHAP⁵
- EAP-TLS – využívá Certifikáty
- EAP-PSK – využívá pre-shared key

11 Bezpečnost bezdrátových technologií

⁴Protokol pro odesílání služebních informací o komunikaci.

⁵Nástupce PAP protokolu.